Composite

Definition: The Composite pattern allows you to compose objects into tree structures to represent part-whole hierarchies. Clients can treat individual/compositions of objects uniformly.

Principle:
- **LSP** – clients can treat leaf and composite uniformly via the `Component` interface.
- **OCP** – you can add new leaf/composite types without changing client code.
- **SRP** - leaf vs composite responsibilities separated

Use Cases:
- Representing hierarchical structures (e.g. file systems, organisation charts, GUI components).
- Implementing recursive algorithms.
- e.g., `File&Directory`, `Task&TaskGroup`

Structure:
- **Component:** Interface or abstract class that defines operations for both leaf and composite objects.
- **Leaf:** Represents individual objects that have no children.
- **Composite:** Contains leaf and/or composite objects and defines operations for them.

Command

Definition/Intent: The Command pattern encapsulates a request/action as an object, thereby allowing parameterization of clients with queues, requests, and operations.

Principle:
- **SRP** – each command class encapsulates one specific action.
- **OCP** – add new commands (new classes) without changing the invoker.
- **DIP** – invoker depends on `Command` interface, not concrete commands.

Use Cases:
- undo/redo, macro commands, logging operations
- Queuing requests or operations for execution.
- Decoupling sender and receiver of requests.

Structure:
- **Command:** Interface/abstract class that declares a method for executing command.
- **Concrete Command:** Classes that implement the Command interface and encapsulate a request as an object.
- **Invoker:** Class that invokes commands without knowing the details of the command or the receiver. (track and executes commands via history)
- **Receiver:** performs the actual work associated with a command. (holds content)

Decorator

Definition: allows behaviour to be added to individual objects dynamically, without affecting the behaviour of other objects from the same class.

Intent: enhancement or augmentation of the basic functionality by wrapping object's responsibilities.

Principle:
- **OCP** – extend behavior by wrapping objects instead of modifying them.
- **SRP** – each decorator adds one focused responsibility; you can combine them.
- **DIP** – clients depend on the abstract component type, not the concrete decorator.
- **LSP** - decorators substitute components.

Use Cases:
- Extending functionality without subclassing.
- Composing objects with different combinations of features(scrolling + border + logging).

Structure:
- **Component:** Interface or abstract class defining the interface for objects that can have responsibilities added to them dynamically.
- **Concrete Component:** Class representing objects to which additional responsibilities can be added.
- **Decorator:** The base class for concrete decorators. Extends/Implements the Component. Can be an abstract class.
- **Concrete Decorator:** Class that adds additional responsibilities to the component.

Proxy

Definition/Intent: The Proxy pattern provides a surrogate or placeholder for another object to control access to real object (lazy load, caching, security).

Principle:
- **SRP** – real subject does core work; proxy handles access control, caching, remote access, etc.
- **OCP** – you can add proxies without changing client or real subject logic.
- **DIP** – clients work with an interface; proxy and real subject both implement it.

Use Cases:
- Expensive remote call, protection, virtual loading
- Lazy initialisation of resources.
- Access control and permissions.
- Logging, monitoring or caching access to objects.

Structure:
- **Subject:** Interface that defines common operations for both the RealSubject and Proxy.
- **RealSubject:** Class that represents the real object being proxied.
- **Proxy:** Class that acts as a surrogate for the RealSubject and controls access to it.

Adapter

Definition: The Adapter pattern allows objects with complex, incompatible interfaces to work together by providing a wrapper with a simple unified,compatible interface so they can work together.

Principle: OCP – you can integrate new/adapted classes without changing client code. DIP – clients depend on the target interface; adapters wrap concrete implementations behind that interface.

ISP - Target interface is small and focused, hiding the large adaptee API from clients. **SRP** – client runs app logic, adaptee logs, adapter only translates interfaces.

Use Cases:
- One central class holding references to subsystems and coordinating them
- Integrating new components with existing systems.
- Reusing existing classes with incompatible interfaces.

Structure:
- Target: Interface or abstract class that the client expects.
- Adaptee: Class with an incompatible interface that needs to be adapted.
- Adapter: Class that implements the Target interface and wraps the Adaptee.

Facade

Definition: The Facade pattern provides one simple, high-level API to a set of interfaces in a complex subsystem, making the subsystem easier to use internally calls lots of other classes/services in the right order.).

Intent: simplification,

Principle: SRP – responsibilities are separated: each subsystem class does its own work, the Facade only coordinates them and offers one high-level entry point. **DIP** – clients depend on the Facade (often an abstraction) instead of many concrete subsystem classes. (Can also help **OCP**: we can change internals of the subsystem without breaking clients.) **ISP** - the client uses a small, focused interface instead of depending on many low-level subsystem methods it doesn't need.

Use Cases:
- Simplifying complex systems or APIs.
- Providing a high-level interface to multiple low-level components.
- Hiding implementation details from clients.

Structure:
- Facade: Class that provides a simplified interface to a complex subsystem.
- Subsystem Classes: Classes that implement functionality and are hidden by the facade.
- Client: Utilises the facade to interact with the subsystem.

Core principles in OOP

- **Inheritance:** Allows one class to inherits data and methods from another. Enabling the reuse of code and the establishment of hierarchical relationships.
- **Encapsulation:** Bundles the data and operations on that data into a single unit. Hiding the internal state and some functionality of an object from the outside world.
- **Polymorphism:** Allows objects to take on different forms or have multiple behaviors, enabling flexibility and versatility in programming.
- **Abstraction:** Hide complex implementation details and only show essential features, reducing complexity and enhancing understanding.

SOLID principles

- **S - Single Responsibility:** A class should have one, and only one, reason to change--God class doing UI + logic + persistence
- **O - O/C:** A module should be open for extension but closed for modification, i.e., modules should be written s.t. they can be extended, without requiring them to be modified.
- **L - Liskov Substitution:** Derived classes must be substitutable for their base classes (without affecting correctness)
- **I - Interface Segregation:** No client should be forced to depend on methods it does not use.
- **D - Dependency Inversion:** Abstractions should not depend on details. Details should depend on abstractions--Class directly instantiates lots of concretes.

Boundary Value Test Design

-Choice of inputs for each parameter:
- **Valid only:** (min, min+1, one nominal value, max - 1, max)
- **Both valid and invalid:** (min - 1, min, min+1, one nominal value, max - 1, max, max + 1)

-How many parameters to test at the boundary value in each case:
- Interaction of parameters **does not produce faults:** one at a time
- Interaction of parameters **can produce faults:** multiple

E.g. (valid only, no interaction):
For each parameter, test its boundaries with others at nominal (0). Also include a "all nominal" test like f(0, 0)

Categories of patterns:

Creational: solve problems related to the reuse of existing code/ classes.
Structural: solve problems related to the flexibility of organizing code and class hierarchies within larger systems.
Behavioural: solve problems related to the interaction between objects.

UML relationship types

- **Dependency:** Temporary interactions between objects of the different classes.
- **Association:** Generic link between two classes, e.g., "knows about"
- **Aggregation:** A weaker form of "has-a" relationship.
- **Composition:** A strong form of "has-a" relationship, where the parts do not exist independently of the whole.

Chain of Responsibility

Definition: The Chain of Responsibility pattern allows multiple objects to handle a request without explicitly specifying the handler.

Intent: decouple senders from receivers by allowing multiple objects a chance to handle a request, passing it along a chain until handled.

Principle:
- **SRP** – each handler deals with one type/ aspect of a request.
- **OCP** – add/reorder handlers in the chain without changing the sender.
- **DIP** – sender depends on the handler interface, not specific handler classes.

Use Cases:
- Sequence of checks/filters
- Decoupling request senders from receivers.
- Implementing a processing pipeline with multiple stages.

Structure:
- **Handler:** Interface or abstract class that defines a method for handling requests and optionally a successor.
- **Concrete Handler:** Classes that implement the Handler interface and handle specific types of requests.

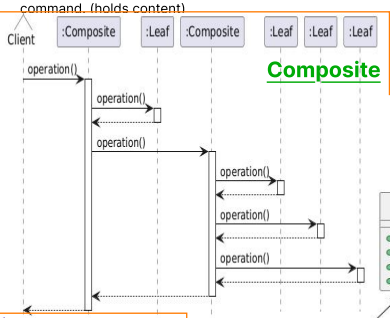
Iterator

Definition/Intent: sequentially access elements of aggregate without exposing internal structure.

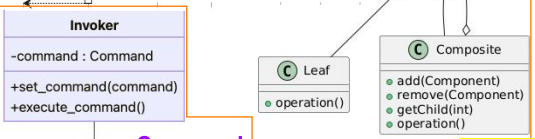
Principle: SRP – separates what is stored from how it is traversed: The Student class only manages the student's enrolled units, while StudentUnitIterator is responsible for traversing those units (e.g., normal, reverse, filtered). Changes to traversal behaviour don't affect how Student stores its units. **ISP** – iterator interface is small and focused (hasNext, next, etc.). **DIP** – clients depend on `Iterator` abstraction, not a concrete collection implementation. **OCP** - new collections provide iterators without changing traversal code.

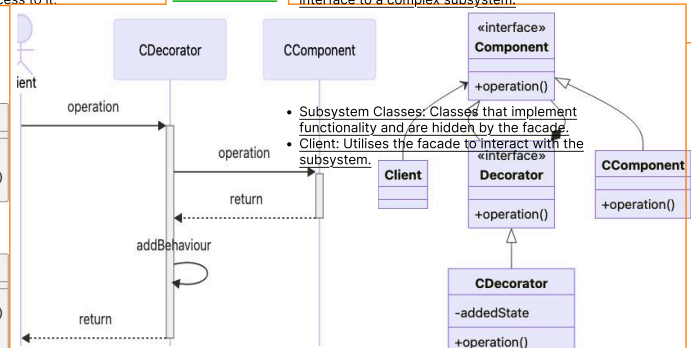
Use Cases:
- You have a collection and need a standard protocol (has_next/next)
- Traversing collections or aggregates without exposing their internal structure.
- Iterating over elements of different types uniformly.
- Implementing custom iterators for complex data structures.

Structure:
- **Iterator:** Interface or abstract class that defines methods for accessing elements of an aggregate object.
- **Concrete Iterator:** Classes that implement the Iterator interface and provide traversal behaviour for specific aggregate types.
- **Aggregate:** Interface or abstract class that defines methods for creating iterators.
- **Concrete Aggregate:** Classes that implement the Aggregate interface and provide specific collections or aggregates.

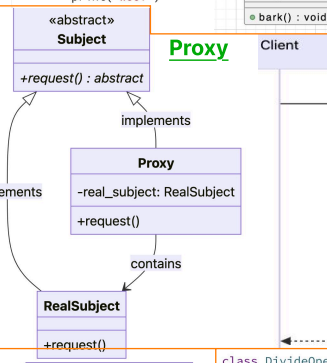


Composite

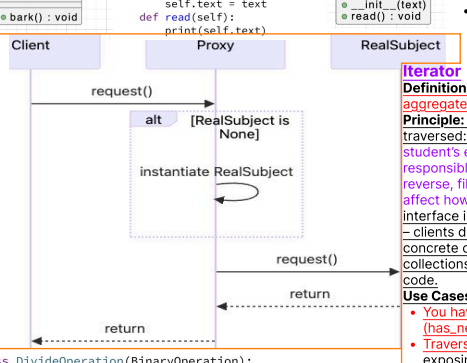


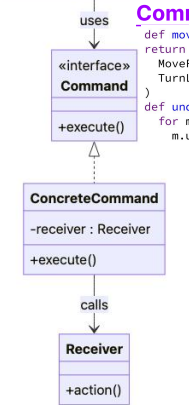


Decorator



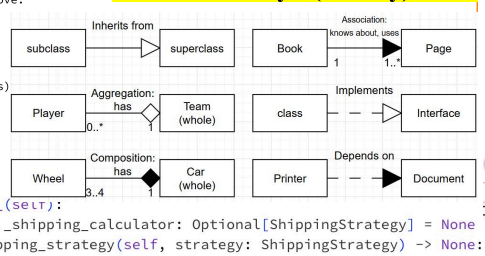
Proxy





Command

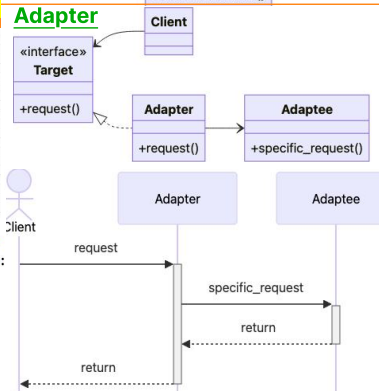
```
def move_diagonally(robot)->Move:
return GroupofMoves(
  MoveForward(robot),
  TurnLeft(robot)
)
def undo(self):
  for m in reversed(self.moves)
    m.undo()
```

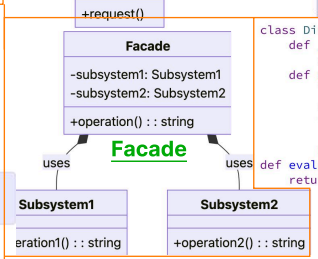



```
class Order:
    def __init__(self):
        self._shipping_calculator: Optional[ShippingStrategy] = None
    def set_shipping_strategy(self, strategy: ShippingStrategy) -> None:
        """set the strategy."""
        self._shipping_calculator = strategy
    def get_shipping_cost(self) -> float:
        if self._shipping_calculator is None:
            raise ValueError("Shipping strategy has not been set.")
        return self._shipping_calculator.calculate_cost(self)
```

Author: Wenyu (Wendy) Fan

Adapter

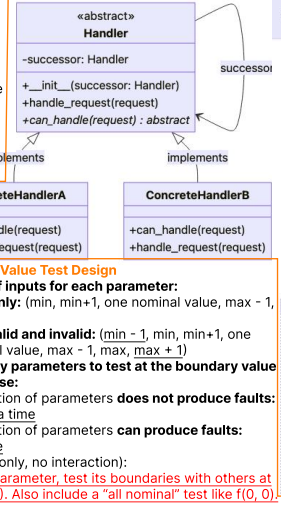


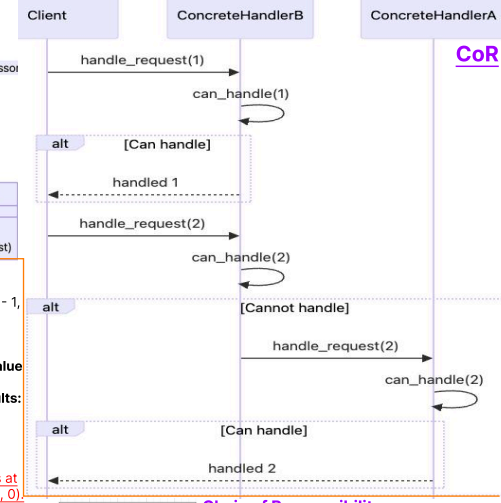


Facade

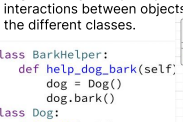
```
class DivideOperation(BinaryOperation):
    def __init__(self, left, right):
        super().__init__(left, right, "/")
    def evaluate(self):
        left_val = self.left.evaluate()
        right_val = self.right.evaluate()
        if right_val == 0:
            raise ValueError("Division by zero")
        return left_val / right_val
def evaluate(expr):
    return expr.evaluate()
```

composite

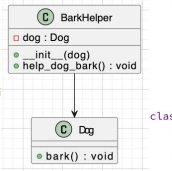




CoR

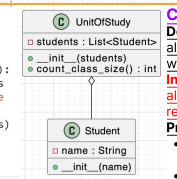



```
class BarkHelper:
    def help_dog_bark(self):
        dog = Dog()
        dog.bark()
class Dog:
    def bark(self):
        print("woof")
```

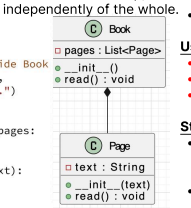



```
class BarkHelper:
    def __init__(self, dog):
        self.dog = dog
    def help_dog_bark(self):
        self.dog.bark() # usi
class Dog:
    def bark(self):
        print("woof")
```

```
class UnitOfStudy:
    def __init__(self, students):
        self.students = students
        # <-- given from outside
    def count_class_size(self):
        return len(self.students)
class Student:
    def __init__(self, name):
        self.name = name
```




```
class Book:
    def __init__(self):
        self.pages = [
        # <-- created inside Book
            Page("On..."),
            Page("Next....")
        ]
    def read(self):
        for page in self.pages:
            page.read()
class Page:
    def __init__(self, text):
        self.text = text
    def read(self):
        print(self.text)
```



V model
-Verification: User Requirements (URS) → Software Requirements (SRS) → Architecture Design → Module Design → Code Implementation
-Validation: Unit Testing ← Integration Testing ← System Testing ← User Acceptance Testing (UAT) ← Execution & Debugging

a. 1. The Client object sends a hasNext() message to the StudentUnitIterator. 2. The iterator responds with false, indicating that there are no more elements to retrieve. 3. No further getNext() messages should be sent once hasNext() has returned false.
b. SRP. Here, the Student class is concerned only with managing the student's enrolled units, while the StudentUnitIterator is responsible for how those units are traversed. This separation means that changes to the traversal behaviour (iterating in reverse or applying filters) would not affect how the Student stores its units.
c. 1. We define a new iterator class, such as AlphabeticalIterator, implementing the same iterator interface. The logic for sorting the data can be implemented in the iterator, and therefore, there are no changes to the existing classes. 2. The Student class can provide a method like create_alphabetical_iterator or create_iterator(order_type) that returns the appropriate iterator instance.

Composite

```python
class Component(ABC):
    @abstractmethod
    def operation(self): ...
class Leaf(Component):
    def operation(self):
        return "Leaf operation"
class Composite(Component):
    def __init__(self):
        self.children = []
    def add(self, component):
        self.children.append(component)
    def remove(self, component):
        self.children.remove(component)
    def operation(self):
        results = []
        for child in self.children:
            results.append(child.operation())
        return "\n".join(results)
# Example Usage:
leaf1 = Leaf()
leaf2 = Leaf()
composite = Composite()
composite.add(leaf1)
composite.add(leaf2)
print(composite.operation())
# Output:
# Leaf operation
# Leaf operation
```

Command

```python
class Move(ABC):
    @abstractmethod
    def execute(self):
        pass
    @abstractmethod
    def undo(self):
        pass
class GroupOfMoves(Move)):
    def __init__(self, moves):
        self.moves = moves
    def execute(self):
        for m in self.moves:
            m.execute()
    def undo(self):
        for m in reversed(self.moves):
            m.undo()
class Receiver:
    def action(self):
        print("Receiver action")
class Invoker:
    def __init__(self):
        self.command = None
    def set_command(self, command):
        self.command = command
    def execute_command(self):
        if self.command:
            self.command.execute()
```

Decorator

```python
class Component(ABC):
    @abstractmethod
    def operation(self): ...
class CComponent(Component):
    def operation(self):
        return "CComponent operation"
class Decorator(Component):
    def __init__(self, component):
        self.component = component
    def operation(self):
        return self.component.operation()
class CDecorator(Decorator):
    def __init__(self, component):
        super().__init__(component)
    def operation(self):
        return f"De operation, {super().operation()}"
# Example Usage:
component = CComponent()
decorator = CDecorator(component)
print(decorator.operation())
# Output: De operation, CComponent op
```

Proxy

```python
class Subject(ABC):
    @abstractmethod
    def request(self):
        pass
class RealSubject(Subject):
    def request(self):
        return "RealSubject's request"
class Proxy(Subject):
    def __init__(self):
        self.real_subject = None
    def request(self):
        if self.real_subject is None:
            self.real_subject = RealSubject()
        return f"Proxy: {self.real_subject.req
# Example Usage:
proxy = Proxy()
print(proxy.request())
# Output: Proxy: RealSubject's request
```

Adapter

```python
class Target:
    def request(self):
        pass
class Adaptee:
    def specific_request(self):
        return "Adaptee's sp request"
class Adapter(Target):
    def __init__(self, adaptee):
        self.adaptee = adaptee
    def request(self):
        res = self.adaptee.specific_request()
        return f"Adapter: {res}"
# Example Usage:
adaptee = Adaptee()
adapter = Adapter(adaptee)
print(adapter.request())
# Output: Adapter: Adaptee's sp request
```

Facade

```python
class Subsystem1:
    def operation1(self):
        return "Subsystem1 operation1"
class Subsystem2:
    def operation2(self):
        return "Subsystem2 operation2"
class Facade:
    def __init__(self):
        self.subsystem1 = Subsystem1()
        self.subsystem2 = Subsystem2()
    def operation(self):
        result = []
        result.append(self.subsystem1.operation1())
        result.append(self.subsystem2.operation2())
        return "\n".join(result)
# Example Usage:
facade = Facade()
print(facade.operation())
# Output:
# Subsystem1 operation
# Subsystem2 operation
```

Observer

```python
class Subject(ABC):
    def __init__(self):
        self.observers = []
    def attach(self, observer):
        self.observers.append(observer)
    def detach(self, observer):
        self.observers.remove(observer)
    def notify(self):
        for observer in self.observers:
            observer.update(self)
    @abstractmethod
    def get_state(self): ...
    @abstractmethod
    def set_state(self, state): ...
class ConcreteSubject(Subject):
    def __init__(self):
        super().__init__()
        self.state = None
    def get_state(self):
        return self.state
    def set_state(self, state):
        self.state = state
        self.notify()
class Observer(ABC):
    @abstractmethod
    def update(self, subject):
        pass
class ConcreteObserverA(Observer):
    def update(self, subject):
        print(f"COA received: {subject.get_state()}")
class ConcreteObserverB(Observer):
    def update(self, subject):
        print(f"COB received: {subject.get_state()}")
# Example Usage:
subject = ConcreteSubject()
observer_a = ConcreteObserverA()
observer_b = ConcreteObserverB()
subject.attach(observer_a)
subject.attach(observer_b)
subject.set_state("New State")
# Output:
# COA received: New State
# COB received: New State
```

Strategy

```python
class Strategy(ABC):
    @abstractmethod
    def execute(self):
        pass
class ConcreteStrategyA(Strategy):
    def execute(self):
        return "ConcreteStrategyA execution"
class ConcreteStrategyB(Strategy):
    def execute(self):
        return "ConcreteStrategyB execution"
class Context:
    def __init__(self, strategy):
        self.strategy = strategy
    def set_strategy(self, strategy):
        self.strategy = strategy
    def execute_strategy(self):
        return self.strategy.execute()
# Example Usage:
strategy_a = ConcreteStrategyA()
strategy_b = ConcreteStrategyB()
context = Context() # Can set starting strategy
context.set_strategy(strategy_a)
print(context.execute_strategy())
# Output: ConcreteStrategyA execution
context.set_strategy(strategy_b)
print(context.execute_strategy())
# Output: ConcreteStrategyB execution
```

Iterator

```python
class Iterator(ABC):
    @abstractmethod
    def has_next(self): ...
    @abstractmethod
    def next(self): ...
class CIterator(Iterator):
    def __init__(self, collection):
        self.collection = collection
        self.index = 0
    def has_next(self):
        return self.index <
               len(self.collection)
    def next(self):
        if self.has_next():
            value = self.collection
                    [self.index]
            self.index += 1
            return value
        else:
            raise StopIteration()
class Aggregate(ABC):
    @abstractmethod
    def create_iterator(self):
        pass
class CAggregate(Aggregate):
    def __init__(self):
        self.data = []
    def add_item(self, item):
        self.data.append(item)
    def create_iterator(self):
        return CIterator(self.data)
# Example Usage:
aggregate = CAggregate()
aggregate.add_item(1)
aggregate.add_item(2)
aggregate.add_item(3)
iterator = aggregate.create_iterator()
while iterator.has_next():
    print(iterator.next())
# Output:
# 1
# 2
# 3
```

CoR

```python
class Handler(ABC):
    def __init__(self, successor=None):
        self.successor = successor
    def handle_request(self, request):
        if self.successor:
            self.successor.handle_request(request)
    @abstractmethod
    def can_handle(self, request):
        pass
class ConcreteHandlerA(Handler):
    def can_handle(self, request):
        return request % 2 == 0
    def handle_request(self, request):
        if self.can_handle(request):
            print(f"Request {request} hby CHA")
        else:
            super().handle_request(request)
class ConcreteHandlerB(Handler):
    def can_handle(self, request):
        return request % 2 != 0
    def handle_request(self, request):
        if self.can_handle(request):
            print(f"Request {request} hby CHB")
        else:
            super().handle_request(request)
# Example Usage:
handler_a = ConcreteHandlerA()
handler_b = ConcreteHandlerB(handler_a)
handler_b.handle_request(1)
handler_b.handle_request(2)
# Output:
# Request 1 handled by CHB
# Request 2 handled by CHA
```

Template

```python
class AbstractClass(ABC):
    def template_method(self):
        self.invariant_method()
        self.overridden_method()
    def invariant_method(self):
        print("Imethod implementation")
    @abstractmethod
    def overridden_method(self): ...
class ConcreteClass(AbstractClass):
    def overridden_method(self):
        print("Cmethod implementation")
# Example Usage:
concrete_object = ConcreteClass()
concrete_object.template_method()
# Output:
# Imethod implementation
# Cmethod implementation
```

Factory

```python
from abc import ABC, abstractmethod
class Product(ABC):
    @abstractmethod
    def operation(self): ...
class CProduct(Product):
    def operation(self):
        return "CProduct Operation"
class Factory:
    @abstractmethod
    def create_product(self):
        pass
class CFactory(Factory):
    def create_product(self):
        return CProduct()
factory = CFactory()
product = factory.create_product()
print(product.operation())
# Output: CProduct Operation
```

Statement: Determine if a test covers a statement

Elements that are **statements**: if / assert / del / pass / return / yield / raise/ import / global / nonlocal / break / continue / for / while / try / with / def

Elements that are **not statements:** else / async / await / lambda

Branch: Determine if a test covers a branch

- if →2 branches; elif →2; else →0; for / while →2
- Branching construct ≠ always a real branch: If a tool can see that a loop/if condition is constant (True, False, etc.), it knows there's only one possible path, so it doesn't count that construct as a branch for coverage.

Subsumption of one coverage criteria with another

- **Branch** coverage subsumes
- **Statement** coverage subsumes
- **Function** coverage.
- 100% statement coverage guarantee 100% branch coverage

Different coverage criteria

-Function Coverage

-Call Coverage: The proportion of function calls invoked vs function calls that exist in the program body (non unique)(consider redundant functions, function pointers, and unreachable calls)

-Class and Module Coverage: How many classes/modules were covered (very coarse)

-Line Coverage

- Counts which physical lines of source code were executed.
- A line is considered covered if any part of it executes, even if not all statements inside are executed.
- Ignores comments and blank lines.
- Some statements may span multiple lines. Multiple statements may be placed on the same line.

-Basic Blocks: A straight-line code sequence with no branches in other than at the entry and no branches out other than at the exit.

Definition about testing

What is software testing

Software process to

- Demonstrate that software meets its requirements
- Find incorrect or undesired behavior caused by faults

Different system properties

- **Functional:** performs all expected functions properly
- **Non-functional:** secure, performance, usability

Objectives of testing

Testing completeness has never been feasible

- So many test cases possible - exhaustive testing is so expensive!
- Risk-driven or risk management strategy to increase our confidence

How much testing is enough?

- Select test cases sufficient for a specific purpose (This class introduces a new technique)

	Normal	Robust
Weak	(best case)	
Strong		(worst case)

What is Basic Blackbox Fuzzing:

- a software testing technique that finds bugs by feeding a program many unexpected, random, or malformed inputs — without using any internal knowledge of the program (no source code, no instrumentation, no coverage feedback). The fuzzer treats the target like a sealed box: give inputs in, observe outputs/crashes/hangs, and repeat.
- In black box fuzzing, the oracles are general. That is, they are not input specific.

Definition of Equivalence Partitioning

- Domain for each parameter is partitioned into regions. Then choose at least one input value from each partition.
  - **Weak** = at least one per partition, no combinations/interaction
  - **Strong** = test combinations of partitions (with interaction)
  - **Normal** = valid only, **Robust** = valid + invalid.

Characteristics of blackbox testing

Key Advantages

- Independent of implementation ⇒ unbiased validation.
- Helps detect missing functionalities.
- End-user focused: aligns with actual usage scenarios.
- Can be applied early, as soon as specifications exist.

Key Limitations

- Cannot guarantee complete coverage of code paths.
- May miss internal defects not reflected in outputs.
- Designing effective tests can be challenging without specs.
- Often requires large number of test cases.

Observer

Definition: The Observer pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Intent: defines a one-to-many dependency between objects so that when one object changes state, all its dependents are automatically notified and updated. Principle:

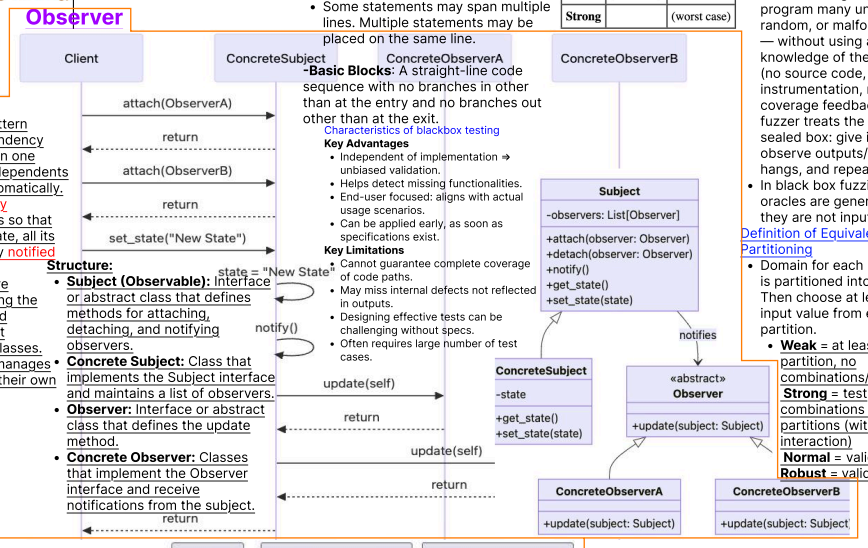
- OCP – you can add/remove observers without changing the subject. DIP – subjects and observers talk via abstract interfaces, not concrete classes. (Also helps SRP: subject manages state; observers manage their own reactions.)

Use Cases:

- Implementing event handling systems.
- GUIs,subscriptions→attach/detach/notify, observers with update()
- Decoupling the subject (observable) from its observers (dependents).

Structure:

- **Subject (Observable):** Interface or abstract class that defines methods for attaching, detaching, and notifying observers.
- **Concrete Subject:** Class that implements the Subject interface and maintains a list of observers.
- **Observer:** Interface or abstract class that defines the update method.
- **Concrete Observer:** Classes that implement the Observer interface and receive notifications from the subject.



Factory Method

Definition/Intent: creating objects that allows subclasses to decide which concrete product to create. Put object creation behind an overridable createX() method.

Use Cases:

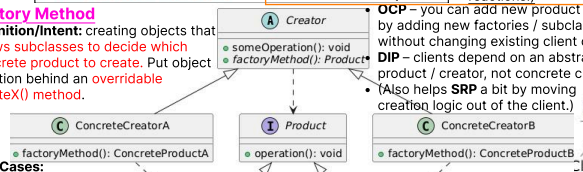
- Decoupling object creation from usage.
- Creating objects based on user input or configuration.
- Don't want big if in base class.

Principle:

- OCP – you can add new product types by adding new factories / subclasses, without changing existing client code.
- DIP – clients depend on an abstract product / creator, not concrete classes.
- (Also helps SRP a bit by moving creation logic out of the client.)

Structure:

- **Product:** Interface or base class for all products.
- **Concrete Products:** Classes implementing the Product interface.
- **Factory (Creator):** Interface or class responsible for creating products.
- **Concrete Factories (Creators):** Classes implementing the Factory interface to create specific products.



Template Method

Definition/Intent: The Template Method pattern defines the skeleton of an algorithm in the superclass but lets subclasses override specific steps of the algorithm without changing its structure.

Principle:

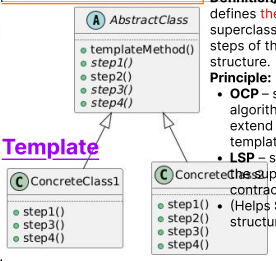
- **OCP** – superclass defines the fixed algorithm; subclasses override steps to extend behavior without changing the template.
- **LSP** – subclasses must be substitutable for the superclass (they honor the algorithm contract).
- (Helps **SRP**: base handles invariant algorithm structure, subclasses handle variable parts.)

Use Cases:

- Several subclasses/algorithms follow same high-level steps but differ in details
- Lift common structure into template in superclass (avoiding duplication), make hooks abstract/overridable

Structure:

- **Abstract Class:** Defines the template method that calls the invariant steps and allows subclasses to override specific steps.
- **Concrete Class:** Subclass of the abstract class that implements the specific steps of the algorithm.



Strategy

Definition/Intent: defines a family of algorithms, encapsulates each one, and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

Principle:

- **OCP** – add new strategies (algorithms) without changing the context.
- **DIP** – context depends on the strategy interface, not concrete implementations.
- **SRP** – each strategy class encapsulates one specific algorithm / behavior.
- **LSP** – strategies interchangeable.

Structure:

- **Strategy:** Interface or abstract class that defines a method for executing the algorithm.
- **Concrete Strategy:** Classes that implement the Strategy interface and provide specific implementations of the algorithm.
- **Context:** Class that maintains a reference to a strategy object and delegates the algorithm's execution to it.

Use Cases:

- Encapsulating algorithms and making them interchangeable.
- Implementing different variations of an algorithm.
- Many "modes" or "methods", big if/elif or switch on a string